Lab Programs

Set 18-22

29/7/26

Name:E.Kavya

Reg NO:192424365

Set 18: Product Inventory Management
Dataset: Product Inventory

Product ID Product Name Quantity Available Price (in $)
1 Product A 250 20
2 Product B 175 15
3 Product C 300 18

1. Create a bar chart to visualize the quantity of each product in the inventory. Label the axes and title the chart.
Code:

```
library(ggplot2)
inventory <- data.frame(
  ProductID = c(1, 2, 3),
  ProductName = c("Product A", "Product B", "Product C"),
  Quantity = c(250, 175, 300),
  Price = c(20, 15, 18)
)
ggplot(inventory, aes(x = ProductName, y = Quantity)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  labs(
    title = "Quantity Available for Each Product",
    x = "Product Name",
    y = "Quantity Available"
  ) +
  theme_minimal()
```

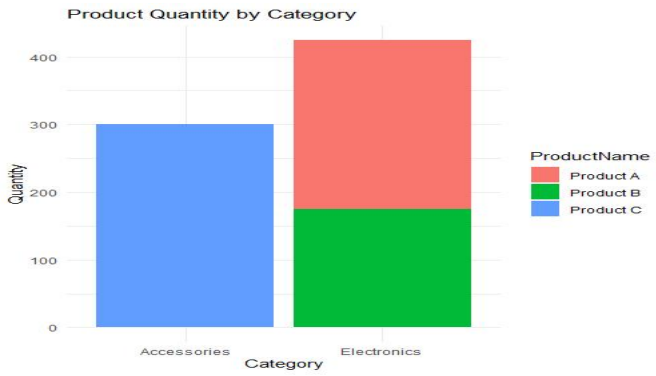
Output:

2. Generate a stacked bar chart to show the quantity of each product within different product categories.

Code:

```
library(ggplot2)
inventory <- data.frame(
  ProductName = c("Product A", "Product B", "Product C"),
  Category = c("Electronics", "Electronics", "Accessories"),
  Quantity = c(250, 175, 300)
)
ggplot(inventory, aes(x = Category, y = Quantity, fill = ProductName)) +
  geom_bar(stat = "identity") +
  labs(
    title = "Product Quantity by Category",
    x = "Category",
    y = "Quantity"
  ) +
  theme_minimal()
```

Output:



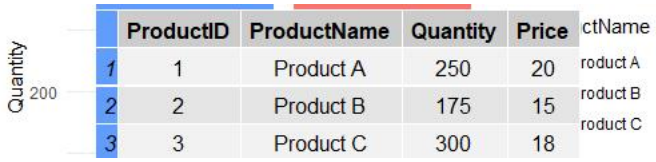
3. Build a table to show the inventory data for each product. Label the table elements.

Code:

```
install.packages("gridExtra")   # Run only once
library(gridExtra)
inventory <- data.frame(
```

```
  ProductID = c(1, 2, 3),
  ProductName = c("Product A", "Product B", "Product C"),
  Quantity = c(250, 175, 300),
  Price = c(20, 15, 18)
)
grid.table(inventory)
```

Output:



	ProductID	ProductName	Quantity	Price
1	1	Product A	250	20
2	2	Product B	175	15
3	3	Product C	300	18

4. Develop a Tableau dashboard combining the bar chart, stacked bar chart, and the table for interactive exploration of inventory data.

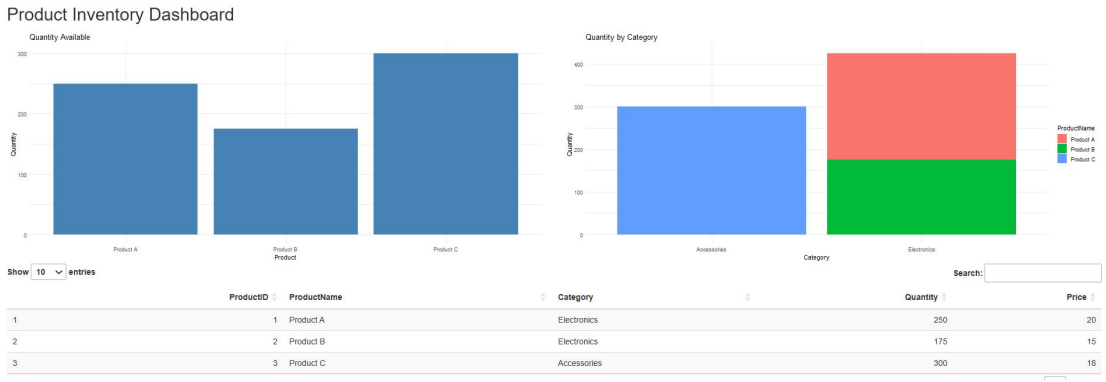
Code:

```
install.packages("shiny")
install.packages("ggplot2")
install.packages("DT")
library(shiny)
library(ggplot2)
library(DT)

inventory <- data.frame(
  ProductID = c(1, 2, 3),
  ProductName = c("Product A", "Product B", "Product C"),
  Category = c("Electronics", "Electronics", "Accessories"),
  Quantity = c(250, 175, 300),
  Price = c(20, 15, 18)
)
ui <- fluidPage(
  titlePanel("Product Inventory Dashboard"),
fluidRow(
    column(6, plotOutput("barChart")),
    column(6, plotOutput("stackedBar"))
  ),
  fluidRow(
    column(12, DTOutput("inventoryTable"))))
server <- function(input, output) {
  output$barChart <- renderPlot({
    ggplot(inventory, aes(x = ProductName, y = Quantity)) +
      geom_bar(stat = "identity", fill = "steelblue") +
      labs(
        title = "Quantity Available",
        x = "Product",
        y = "Quantity"
      ) +
      theme_minimal()
  })
  output$stackedBar <- renderPlot({
```

```
    ggplot(inventory, aes(x = Category, y = Quantity, fill = ProductName)) +
      geom_bar(stat = "identity") +
      labs(
        title = "Quantity by Category",
        x = "Category",
        y = "Quantity"
      ) +
      theme_minimal()
  })
  output$inventoryTable <- renderDT({
    datatable(inventory)
  })
}
shinyApp(ui = ui, server = server)
```

Output:



Set 19: Survey Responses Analysis
Dataset: Survey Responses

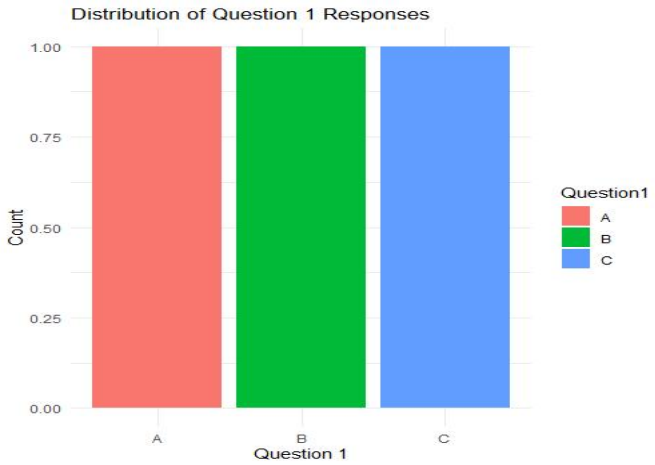
Survey ID	Question 1	Question 2	Question 3
1	A	B	C
2	B	A	D
3	C	A	B

1. Create a grouped bar chart to visualize the distribution of answers for Question 1. Label the chartelements.
Code:

```
library(ggplot2)
survey <- data.frame(
  SurveyID = c(1,2,3),
  Question1 = c("A","B","C"),
  Question2 = c("B","A","A"),
  Question3 = c("C","D","B")
)
ggplot(survey, aes(x = Question1, fill = Question1)) +
  geom_bar(position = "dodge") +
  labs(
    title = "Distribution of Question 1 Responses",
    x = "Question 1",
```

```
    y = "Count"
  ) +
  theme_minimal()
```

Output:

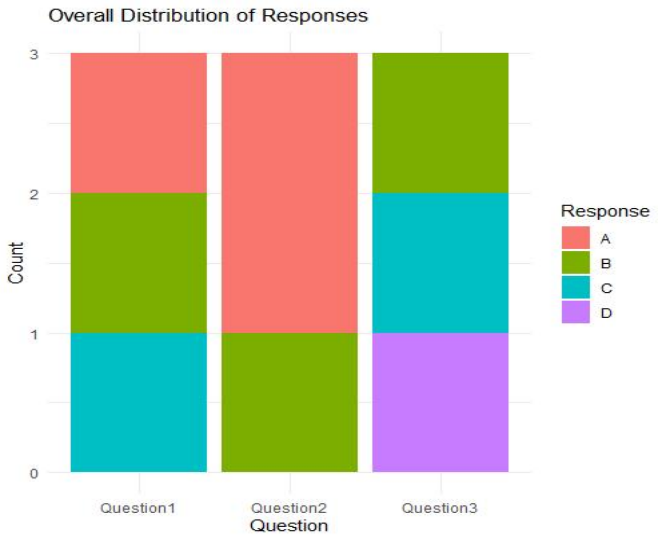


2. Generate a stacked bar chart to represent the overall distribution of responses for all three questions.

Code:

```
library(ggplot2)
library(tidyr)
survey <- data.frame(
  SurveyID = c(1,2,3),
  Question1 = c("A","B","C"),
  Question2 = c("B","A","A"),
  Question3 = c("C","D","B")
)
survey_long <- pivot_longer(
  survey,
  cols = Question1:Question3,
  names_to = "Question",
  values_to = "Response"
)
ggplot(survey_long, aes(x = Question, fill = Response)) +
  geom_bar() +
  labs(
    title = "Overall Distribution of Responses",
    x = "Question",
    y = "Count"
  ) +
  theme_minimal()
```

Output:



3.Build a table to show the survey response data for each survey. Label the table elements.

Code:

```
library(DT)
survey <- data.frame(
  SurveyID = c(1,2,3),
  Question1 = c("A","B","C"),
  Question2 = c("B","A","A"),
  Question3 = c("C","D","B")
)
datatable(
  survey,
  colnames = c("Survey ID","Question 1","Question 2","Question 3")
)
```

Output:

Show 10 entries

Search:

	Survey ID	Question 1	Question 2	Question 3
1	1	A	B	C
2	2	B	A	D
3	3	C	A	B

4. Develop a Tableau dashboard combining the grouped bar chart, stacked bar chart, and the table for interactive exploration of survey responses.

Code:

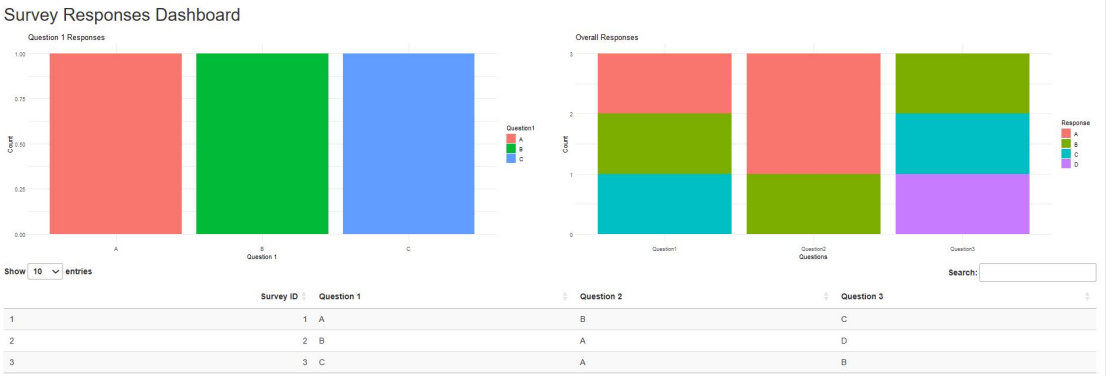
```
library(shiny)
library(ggplot2)
library(tidyr)
```

```
library(DT)
survey <- data.frame(
  SurveyID = c(1,2,3),
  Question1 = c("A","B","C"),
  Question2 = c("B","A","A"),
  Question3 = c("C","D","B")
)
survey_long <- pivot_longer(
  survey,
  cols = Question1:Question3,
  names_to = "Question",
  values_to = "Response"
)
ui <- fluidPage(
  titlePanel("Survey Responses Dashboard"),
  fluidRow(
    column(6, plotOutput("grouped")),
    column(6, plotOutput("stacked"))
  ),
  fluidRow(
    column(12, DTOutput("table"))
  )
)

server <- function(input, output) {
 output$grouped <- renderPlot({
    ggplot(survey, aes(x = Question1, fill = Question1)) +
      geom_bar(position = "dodge") +
      labs(
        title = "Question 1 Responses",
        x = "Question 1",
        y = "Count"
      ) +
      theme_minimal()
  })
 output$stacked <- renderPlot({
    ggplot(survey_long, aes(x = Question, fill = Response)) +
      geom_bar() +
      labs(
        title = "Overall Responses",
        x = "Questions",
        y = "Count"
      ) +
      theme_minimal()
  })
  output$table <- renderDT({
    datatable(
      survey,
      colnames = c("Survey ID","Question 1","Question 2","Question 3")
    )
```

```
  })}
shinyApp(ui = ui, server = server)
```

Output:



Set 20: Stock Analysis
Dataset: Stock Prices

Date Stock A Stock B Stock C
2023-01-01 100 150 120
2023-01-02 105 152 118
2023-01-03 110 148 122

1. Create a line chart to visualize the stock prices of three companies (Stock A, Stock B, and StockC) over a specific time period. Label the axes and title the chart.
Code:

```
library(ggplot2)
library(tidyr)
stock <- data.frame(
  Date = as.Date(c("2023-01-01","2023-01-02","2023-01-03")),
  StockA = c(100,105,110),
  StockB = c(150,152,148),
  StockC = c(120,118,122)
)
stock_long <- pivot_longer(
  stock,
  cols = StockA:StockC,
  names_to = "Company",
  values_to = "Price"
)ggplot(stock_long, aes(x = Date, y = Price, color = Company, group = Company)) +
  geom_line(size = 1) +
  geom_point(size = 3) +
  labs(
    title = "Stock Prices Over Time",
    x = "Date",
    y = "Stock Price"
  ) +
  theme_minimal()
```

Output:

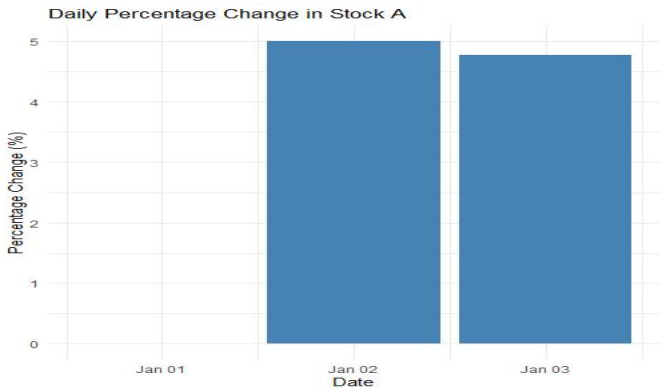
2. Generate a bar chart showing the daily percentage change in stock prices for Stock A. Label the chart elements.

Code:

```
library(ggplot2)
stock <- data.frame(
  Date = as.Date(c("2023-01-01","2023-01-02","2023-01-03")),
  StockA = c(100,105,110)
)

stock$PercentageChange <- c(
  0,
  diff(stock$StockA) / head(stock$StockA,-1) * 100
)
ggplot(stock, aes(x = Date, y = PercentageChange)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  labs(
    title = "Daily Percentage Change in Stock A",
    x = "Date",
    y = "Percentage Change (%)"
  ) +
  theme_minimal()
```

Output:



3. Build a table to display the stock price data for each company over the given period. Label the table elements.

Code:

```
library(DT)
stock <- data.frame(
  Date = as.Date(c("2023-01-01","2023-01-02","2023-01-03")),
  StockA = c(100,105,110),
  StockB = c(150,152,148),
  StockC = c(120,118,122)
)
datatable(
  stock,
  colnames = c("Date","Stock A","Stock B","Stock C")
)
```

Output:

	Date	Stock A	Stock B	Stock C
1	2023-01-01	100	150	120
2	2023-01-02	105	152	118
3	2023-01-03	110	148	122

4. Develop a Tableau dashboard combining the line chart, bar chart, and the table for interactive exploration of stock prices.

Code:

```
library(shiny)
library(ggplot2)
library(tidyr)
library(DT)
stock <- data.frame(
  Date = as.Date(c("2023-01-01","2023-01-02","2023-01-03")),
  StockA = c(100,105,110),
  StockB = c(150,152,148),
  StockC = c(120,118,122)
)
stock_long <- pivot_longer(
  stock,
  cols = StockA:StockC,
  names_to = "Company",
  values_to = "Price"
)
stock$PercentageChange <- c(
  0,
  diff(stock$StockA) / head(stock$StockA,-1) * 100
)
ui <- fluidPage(
  titlePanel("Stock Analysis Dashboard"),
  fluidRow(
    column(6, plotOutput("lineChart")),
    column(6, plotOutput("barChart"))
```

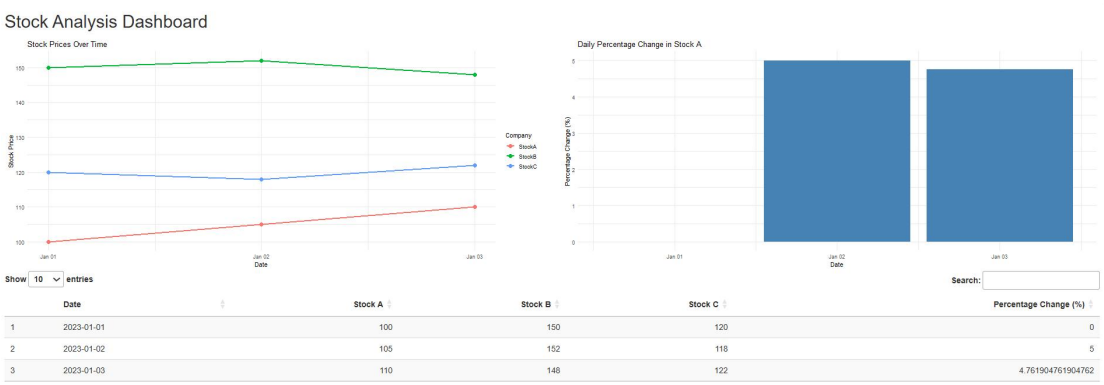
```
  ),
  fluidRow(
    column(12, DTOutput("stockTable"))
  ))
server <- function(input, output) {
  output$lineChart <- renderPlot({
    ggplot(stock_long, aes(x = Date, y = Price, color = Company, group = Company))
+
      geom_line(size = 1) +
      geom_point(size = 3) +
      labs(
        title = "Stock Prices Over Time",
        x = "Date",
        y = "Stock Price"
      ) +
      theme_minimal()
  })
  output$barChart <- renderPlot({
    ggplot(stock, aes(x = Date, y = PercentageChange)) +
      geom_bar(stat = "identity", fill = "steelblue") +
      labs(
        title = "Daily Percentage Change in Stock A",
        x = "Date",
        y = "Percentage Change (%)"
      ) +
      theme_minimal()
  })

  output$stockTable <- renderDT({
    datatable(
      stock,
      colnames = c("Date","Stock A","Stock B","Stock C","Percentage Change (%)")
    )
  })

}

shinyApp(ui = ui, server = server)
```

Output:



Set 21

Dataset: Energy_Consumption_Data.csv

Dataset: Energy_Consumption_Data.csv

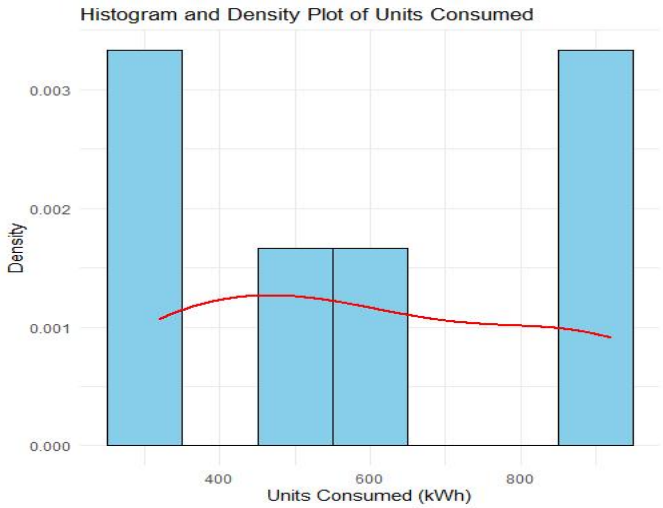
Sector	Region	Month	Temperature (°C)	Units_Consumed (kWh)	Cost (₹)	Renewable_Usage (%)	Peak_Hours
Residential	North	Jan	15	320	2100	22	4
Commercial	South	Jan	24	540	3600	18	6
Industrial	West	Feb	20	880	5900	12	8
Residential	East	Feb	18	350	2300	25	5
Commercial	North	Mar	28	610	4100	20	7
Industrial	South	Mar	30	920	6200	15	9

1. Import the dataset in R. Plot a histogram and density plot for Units_Consumed. Compare consumption patterns.

Code:

```
library(ggplot2)
energy <- data.frame(
  Sector =
c("Residential","Commercial","Industrial","Residential","Commercial","Industrial"),
  Region = c("North","South","West","East","North","South"),
  Month = c("Jan","Jan","Feb","Feb","Mar","Mar"),
  Temperature = c(15,24,20,18,28,30),
  Units_Consumed = c(320,540,880,350,610,920),
  Cost = c(2100,3600,5900,2300,4100,6200),
  Renewable_Usage = c(22,18,12,25,20,15),
  Peak_Hours = c(4,6,8,5,7,9)
)
ggplot(energy, aes(x = Units_Consumed)) +
  geom_histogram(aes(y = after_stat(density)), binwidth = 100, fill = "skyblue", color
= "black") +
  geom_density(color = "red", linewidth = 1) +
  labs(
    title = "Histogram and Density Plot of Units Consumed",
    x = "Units Consumed (kWh)",
    y = "Density"
  ) +
  theme_minimal()
```

Output:

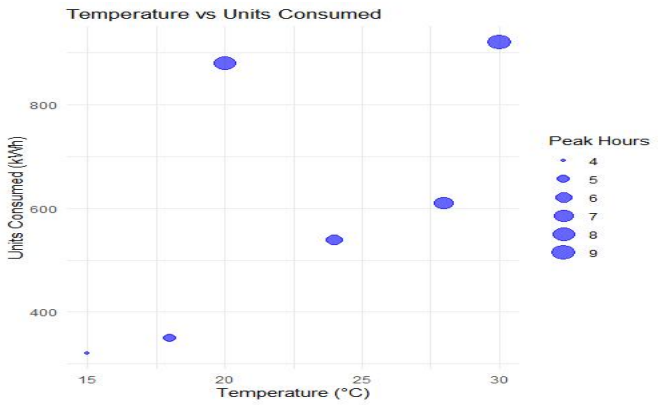


2. Using R programming, create a scatter plot between Temperature and Units_Consumed. Represent Peak_Hours using bubble size. Apply transparency.

Code:

```
library(ggplot2)
energy <- data.frame(
  Sector =
c("Residential","Commercial","Industrial","Residential","Commercial","Industrial"),
  Region = c("North","South","West","East","North","South"),
  Month = c("Jan","Jan","Feb","Feb","Mar","Mar"),
  Temperature = c(15,24,20,18,28,30),
  Units_Consumed = c(320,540,880,350,610,920),
  Cost = c(2100,3600,5900,2300,4100,6200),
  Renewable_Usage = c(22,18,12,25,20,15),
  Peak_Hours = c(4,6,8,5,7,9)
)
ggplot(energy, aes(x = Temperature, y = Units_Consumed, size = Peak_Hours)) +
  geom_point(alpha = 0.6, color = "blue") +
  labs(
    title = "Temperature vs Units Consumed",
    x = "Temperature (°C)",
    y = "Units Consumed (kWh)",
    size = "Peak Hours"
  ) +
  theme_minimal()
```

Output:

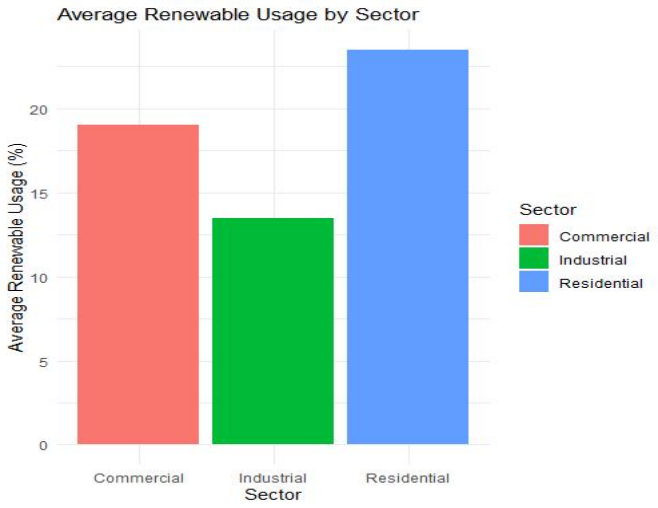


3. Using R, calculate average Renewable_Usage for each Sector. Create a bar chart. Interpret renewable energy adoption.

Code:

```
library(ggplot2)
energy <- data.frame(
  Sector =
c("Residential","Commercial","Industrial","Residential","Commercial","Industrial"),
  Region = c("North","South","West","East","North","South"),
  Month = c("Jan","Jan","Feb","Feb","Mar","Mar"),
  Temperature = c(15,24,20,18,28,30),
  Units_Consumed = c(320,540,880,350,610,920),
  Cost = c(2100,3600,5900,2300,4100,6200),
  Renewable_Usage = c(22,18,12,25,20,15),
  Peak_Hours = c(4,6,8,5,7,9)
)
avg_usage <- aggregate(Renewable_Usage ~ Sector, data = energy, mean)
ggplot(avg_usage, aes(x = Sector, y = Renewable_Usage, fill = Sector)) +
  geom_bar(stat = "identity") +
  labs(
    title = "Average Renewable Usage by Sector",
    x = "Sector",
    y = "Average Renewable Usage (%)"
  ) +
  theme_minimal()
```

Output:



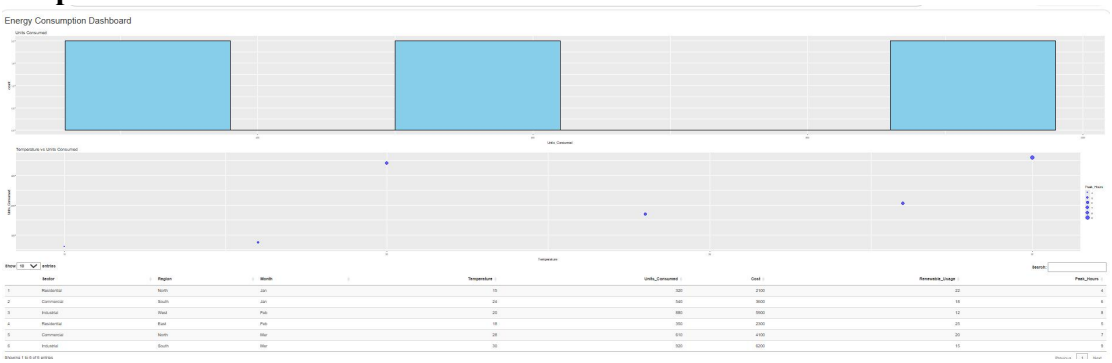
4. Import Energy_Consumption_Data.csv into Tableau. Create a line chart for monthly energy usage trends. Create a heatmap (Sector vs Region vs Units). Add filters for Month and Sector.Design an interactive dashboard.

Code:

```
library(shiny)
library(ggplot2)
library(DT)
energy <- data.frame(
  Sector =
c("Residential","Commercial","Industrial","Residential","Commercial","Industrial"),
  Region = c("North","South","West","East","North","South"),
  Month = c("Jan","Jan","Feb","Feb","Mar","Mar"),
  Temperature = c(15,24,20,18,28,30),
  Units_Consumed = c(320,540,880,350,610,920),
  Cost = c(2100,3600,5900,2300,4100,6200),
  Renewable_Usage = c(22,18,12,25,20,15),
  Peak_Hours = c(4,6,8,5,7,9)
)
ui <- fluidPage(
  titlePanel("Energy Consumption Dashboard"),
  plotOutput("hist"),
  plotOutput("scatter"),
  DTOutput("table")
)
server <- function(input, output) {
  output$hist <- renderPlot({
    ggplot(energy, aes(x = Units_Consumed)) +
      geom_histogram(fill = "skyblue", color = "black", bins = 6) +
      labs(title = "Units Consumed")
  })
  output$scatter <- renderPlot({
    ggplot(energy, aes(x = Temperature, y = Units_Consumed, size = Peak_Hours)) +
      geom_point(alpha = 0.6, color = "blue") +
      labs(title = "Temperature vs Units Consumed")
  })
```

```
  output$table <- renderDT({
    datatable(energy)
  })
}
shinyApp(ui, server)
```

Output:



Set 22: Time Series Analysis
Dataset: Monthly Sales Data

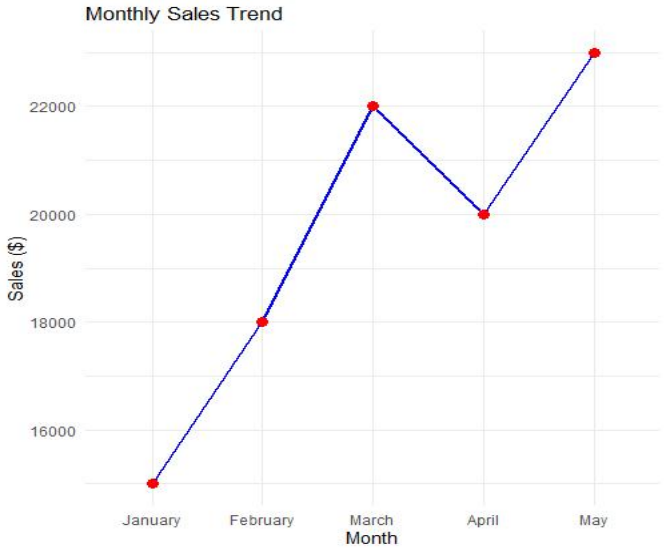
Month Sales (in $)
January 15000
February 18000
March 22000
April 20000
May 23000

1. In R, create a time series line chart to visualize the trend in monthly sales. Label the axes and title the chart.

Code:

```
library(ggplot2)
sales <- data.frame(
  Month = factor(c("January","February","March","April","May"),
                 levels = c("January","February","March","April","May")),
  Sales = c(15000,18000,22000,20000,23000)
)
ggplot(sales, aes(x = Month, y = Sales, group = 1)) +
  geom_line(color = "blue", linewidth = 1) +
  geom_point(color = "red", size = 3) +
  labs(
    title = "Monthly Sales Trend",
    x = "Month",
    y = "Sales ($)"
  ) +
  theme_minimal()
```

Output:

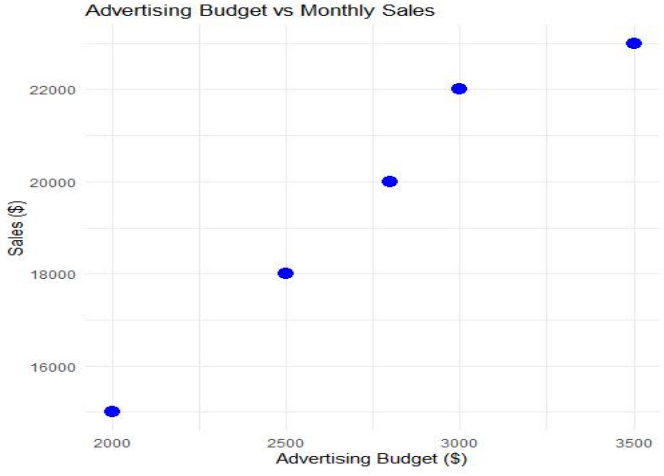


2. In R, generate a scatter plot to analyze the relationship between advertising budget and monthly sales. Explain any insights.

Code:

```
library(ggplot2)
sales <- data.frame(
  Month = c("January","February","March","April","May"),
  Advertising_Budget = c(2000,2500,3000,2800,3500),
  Sales = c(15000,18000,22000,20000,23000)
)
ggplot(sales, aes(x = Advertising_Budget, y = Sales)) +
  geom_point(color = "blue", size = 4) +
  labs(
    title = "Advertising Budget vs Monthly Sales",
    x = "Advertising Budget ($)",
    y = "Sales ($)"
  ) +
  theme_minimal()
```

Output:



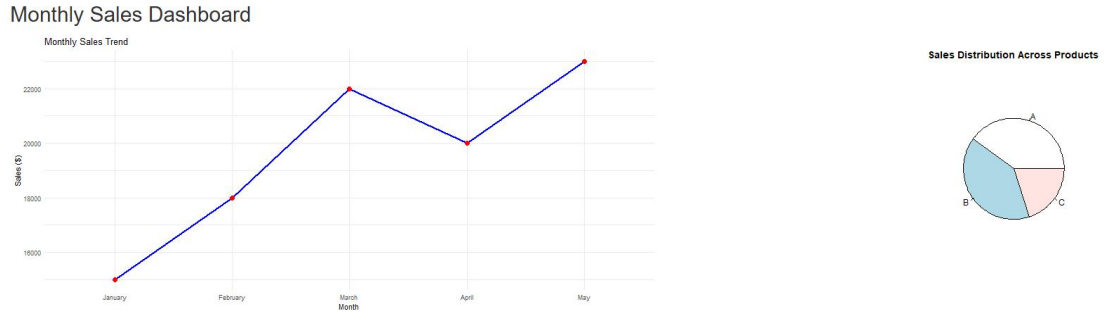
3. In Tableau, build a dashboard combining a line chart showing sales trend and a pie chart displaying the distribution of sales across products.

Code:

```
library(shiny)
library(ggplot2)
sales <- data.frame(
  Month = factor(c("January","February","March","April","May"),
                 levels = c("January","February","March","April","May")),
  Sales = c(15000,18000,22000,20000,23000),
  Product = c("A","B","C","A","B")
)
ui <- fluidPage(
  titlePanel("Monthly Sales Dashboard"),
  fluidRow(
    column(6, plotOutput("lineChart")),
    column(6, plotOutput("pieChart"))
  )
)
server <- function(input, output){
  output$lineChart <- renderPlot({
    ggplot(sales, aes(x = Month, y = Sales, group = 1)) +
      geom_line(color = "blue", linewidth = 1) +
      geom_point(color = "red", size = 3) +
      labs(title = "Monthly Sales Trend", x = "Month", y = "Sales ($)") +
      theme_minimal()
  })
  output$pieChart <- renderPlot({
    pie(table(sales$Product), main = "Sales Distribution Across Products")
  })
}

shinyApp(ui, server)
```

Output:



4. In R, create an autocorrelation plot to identify seasonality in the time series data.

Code:

```
sales_ts <- ts(c(15000,18000,22000,20000,23000), frequency = 12)
```

```
acf(sales_ts,
    main = "Autocorrelation Plot of Monthly Sales")
```

Output: